Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

Smart Courier Delivery and Route Optimization System

Team Name: *Team #*	*Route Rangers* *DSCPP-III-2026-T332* *Section: J and ML4*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Rai, Arpit – 2510012741* *arpitrai012@gmail.com* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Vardhan, Shreshth – 2510379364* *vardhanshreshth@gmail.com* 
Mentor:	*Ms. Isha Deshmukh*

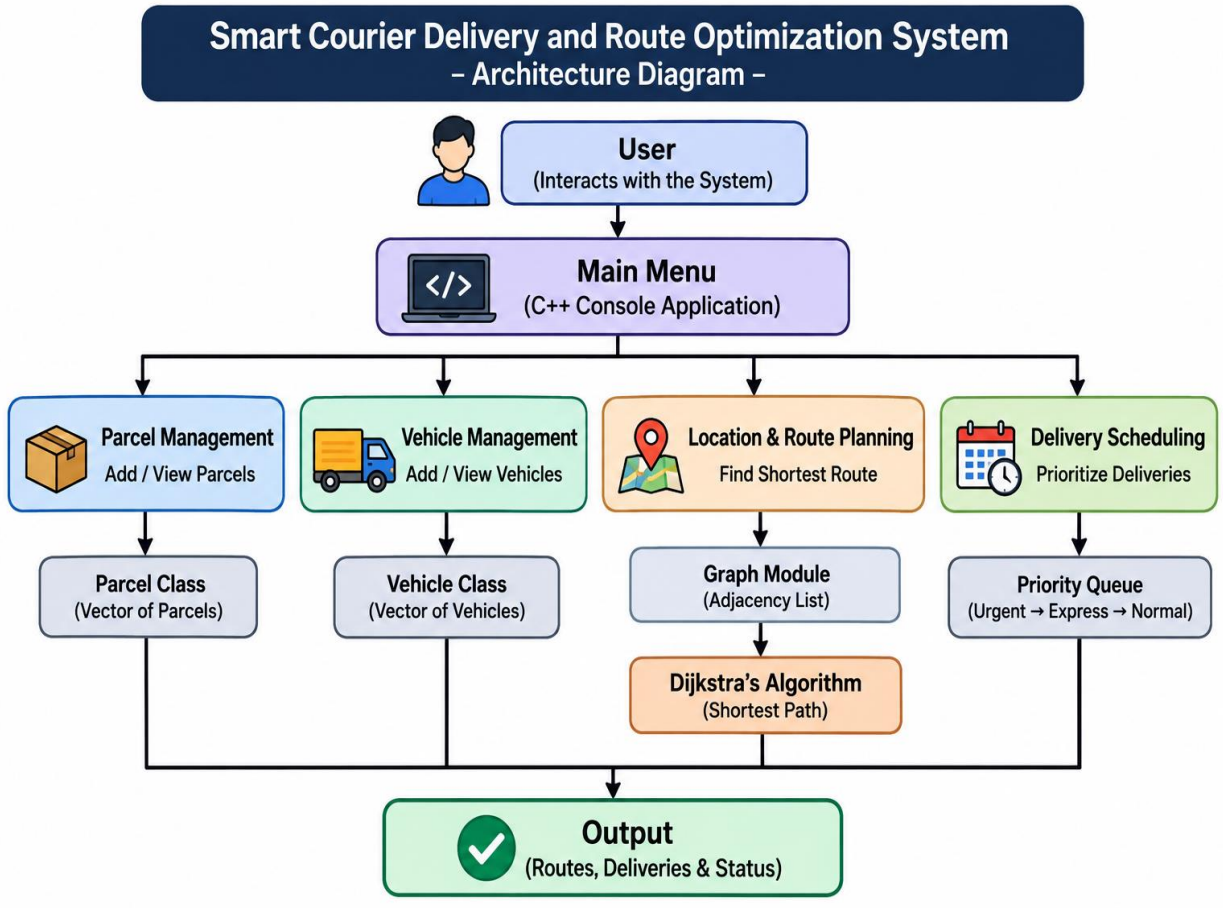
PHASE-III PROJECT PROGRESS DESCRIPTION

Project Abstract

The Smart Courier Delivery and Route Optimization System is a C++ console application planned to help organize parcel deliveries more efficiently. The user can register parcels and vehicles, record parcel weight and delivery priority, and create a small road network by adding locations and distances between them. The system will use data structures and algorithms to find useful routes and organize deliveries. A weighted graph represented with an adjacency list will model the road network, while Dijkstra's algorithm will find the shortest path between two locations. A priority queue will help process urgent parcels before express and normal parcels. Vehicle-capacity checks will prevent assigning more parcel weight than a vehicle can carry. The project is intended as an educational simulation using sample locations and distances, not a live navigation service.

Updated Project Approach and Architecture

The project is developed in C++ as a menu-driven console application in Visual Studio Code, using a GCC/MinGW compiler. It does not require a network connection or communication protocol because all inputs are entered locally by the user. The program is organized around Parcel and Vehicle classes. These classes keep related information together and demonstrate object-oriented programming concepts such as encapsulation, constructors and member functions. Vectors from the C++ Standard Library store collections of parcels and vehicles. The planned Graph module will store locations and weighted roads in an adjacency list. Dijkstra's algorithm will use a priority queue to calculate a shortest path. A separate delivery-priority queue will rank parcels by urgency. Vehicle capacity validation will compare the total parcel weight with the selected vehicle's capacity. Streams is used to save and load project data. The implementation is developed in small modules and tested after each feature is connected.



Tasks Completed

Task Completed	Team Member
Parcel and vehicle classes; add/view operations and menu integration.	Arpit Rai
Selected the courier delivery and route optimization problem and defined its main purpose.	Both members
Dijkstra shortest-path calculation and route display with priority-queue delivery scheduling and processing.	Shreshth Vardhan
Integration and testing of core menu options.	Both members

Challenges/Roadblocks

During development, we had to connect separate modules and keep the menu, parcel/vehicle records, graph, and delivery queue working together. One challenge is connecting the different algorithms into one program without making the code difficult to manage. The basic parcel and vehicle classes are a starting point, but the graph, shortest-path calculation and route-improvement stages still need to be integrated and tested. Another challenge is making sure the route calculation uses valid location IDs and that the system handles cases where no route exists. The program must also reject invalid weights, priorities and vehicle capacities. During the first compilation attempts, a linker/build error was encountered, so the build process needs to be checked before adding more modules. We have solved these issues by compiling after each change, testing with a small road network first, and checking both normal and invalid inputs. The route optimization algorithms will be tested with hand-calculated examples so we can compare their output and explain why a route was selected. We had also kept the code split into clear classes and header files.

Project Outcome/Deliverables

The current deliverable is a working C++ console application for a courier delivery scenario. It will allow users to add and view parcels and vehicles, represent roads between locations, calculate shortest routes, prioritize urgent deliveries and check vehicle load capacity. The advanced route-planning stage will compare possible delivery combinations using the Clarke–Wright savings method and attempt to improve stop order with 2-Opt. The project also includes sample input data, saved project records, source code, and test cases. The final demonstration shows how vectors, queues, priority queues, graphs, searching/sorting where applicable, file handling, classes and constructors are used to solve a practical problem.

Progress Overview

The main program structure and core modules have been implemented and tested such as parcel/vehicle management, these parts form the foundation and its application is finished or fully tested. The road graph, Dijkstra shortest-path feature, delivery-priority queue, vehicle-capacity assignment, route construction, 2-Opt improvement and file persistence are all completed. Some features is being marked as not fully complete until it compiles and passes a repeatable test. Advanced route optimization and final documentation took the most time, so the team divided the modules and integrate them gradually.

Codebase Information

Repository: https://github.com/Anonymous0-27/Smart-Courier-Route-Optimizer.git Main branch: main
The repository contains the C++ project source and README. The work is divided across parcel/vehicle management, graph and Dijkstra routing, and priority scheduling.

Data Structures Used:

- **Vector:** Stores parcel and vehicle records.
- **Graph:** Represents delivery locations and roads with distances.
- **Priority Queue:** Helps process urgent deliveries before normal deliveries.

Algorithms Planned:

- **Dijkstra's Algorithm:** Finds the shortest path between two delivery locations.
- **Clarke–Wright Savings Algorithm:** Helps plan efficient routes for multiple deliveries.
- **2-Opt Algorithm:** Improves a delivery route by reducing its total distance.

Object-Oriented Programming:
The project uses classes, objects, constructors, encapsulation, and member functions to organize parcel and vehicle information.

Testing and Validation Status

Test Type	Status (Pass/Fail)	Notes
Source review	Pass	Parcel and Vehicle class structure reviewed.
Parcel/vehicle add and view	Pass	Program builds successfully.
Priority delivery queue/ Shortest route	Pass	Program builds successfully.
Graph and route algorithms	Pass	Comprehensive validation implemented.

Deliverables Progress

Project design and class outline. Completed

Parcel and vehicle menu. Completed

Weighted graph and Dijkstra. Completed

Priority delivery and capacity validation. Completed

Clarke-Wright and 2-Opt route optimization. Completed

File saving/loading. Completed